



Network Intrusion Detection System (NIDS): Implementing with Snort

Course Objectives

1 Understand NIDS 2 Explore Snort

Learn how Network Intrusion Detection Systems work.

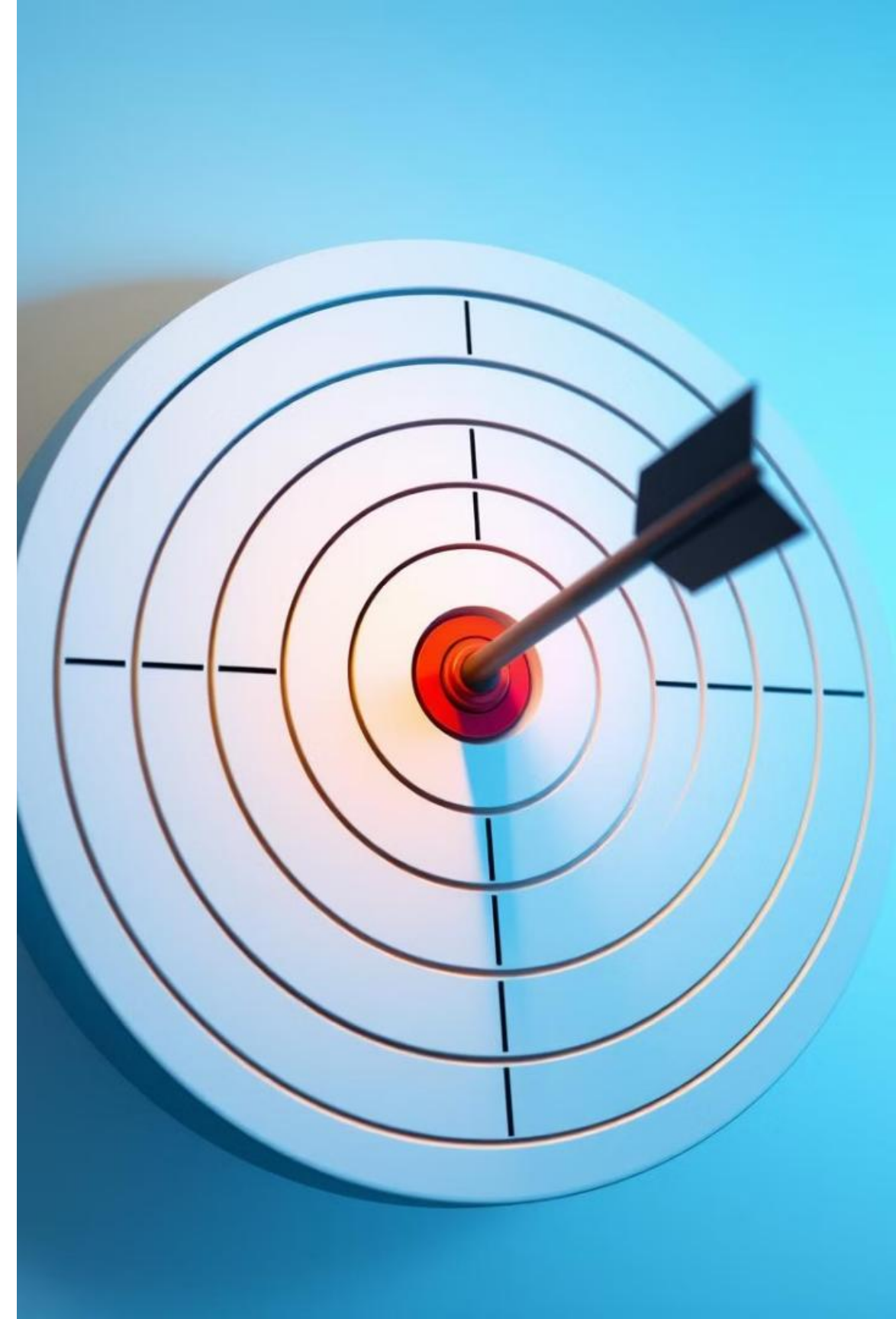
Understand Snort and its different operation modes.

3 Understand and Write Snort Rules

Create and test effective detection rules.

4 Snort Hands-On Lab

Step-by-Step Exercises Using Snort for Threat Detection



1. HIDS vs NIDS ?

HIDS:

Monitors Logs on a specific endpoint device, generating alerts when a signature is detected.

NIDS:

Monitors traffic across the entire network, looking for signatures and generating alerts when identified.

1. HIPS vs NIPS ?

HIPS:

HIDS + Reactions

NIPS:

NIDS + Reactions

NIDS/NIPS Detection Methods

- Signature-based: detects **known threats** by matching traffic against attack patterns, offering low false positives but failing against zero-day attacks.
- Anomaly-based: uses ML/statistical models **to identify deviations from normal behavior**, detecting zero-day attacks but often generating more false positives.
- Hybrid approaches: Hybrid NIDS **merges** signature and anomaly-based detection, improving threat coverage but requiring more complex setup and management.

HIDS/NIDS, Same Detection Methods, BUT Different Context

The core detection methods are indeed similar between HIDS and NIDS. **But the data source difference leads** to critical distinctions in what they can detect and how they operate.

Question !

What can **HIDS** detect that **NIDS** might miss, and vice versa?

And how do we solve this problem?

Question 2 !

An attacker uses an **HTTPS (encrypted) connection** to exfiltrate sensitive data from a corporate server. The network is protected by a **signature-based NIDS**.

Why would this NIDS fail to detect the attack? How could an **anomaly-based NIDS** partially address this issue—and what would its limitations be?

Which network security solution should be implemented?

Cloudflare

Scalable cloud solution for enterprises, but requires a paid subscription.

Suricata

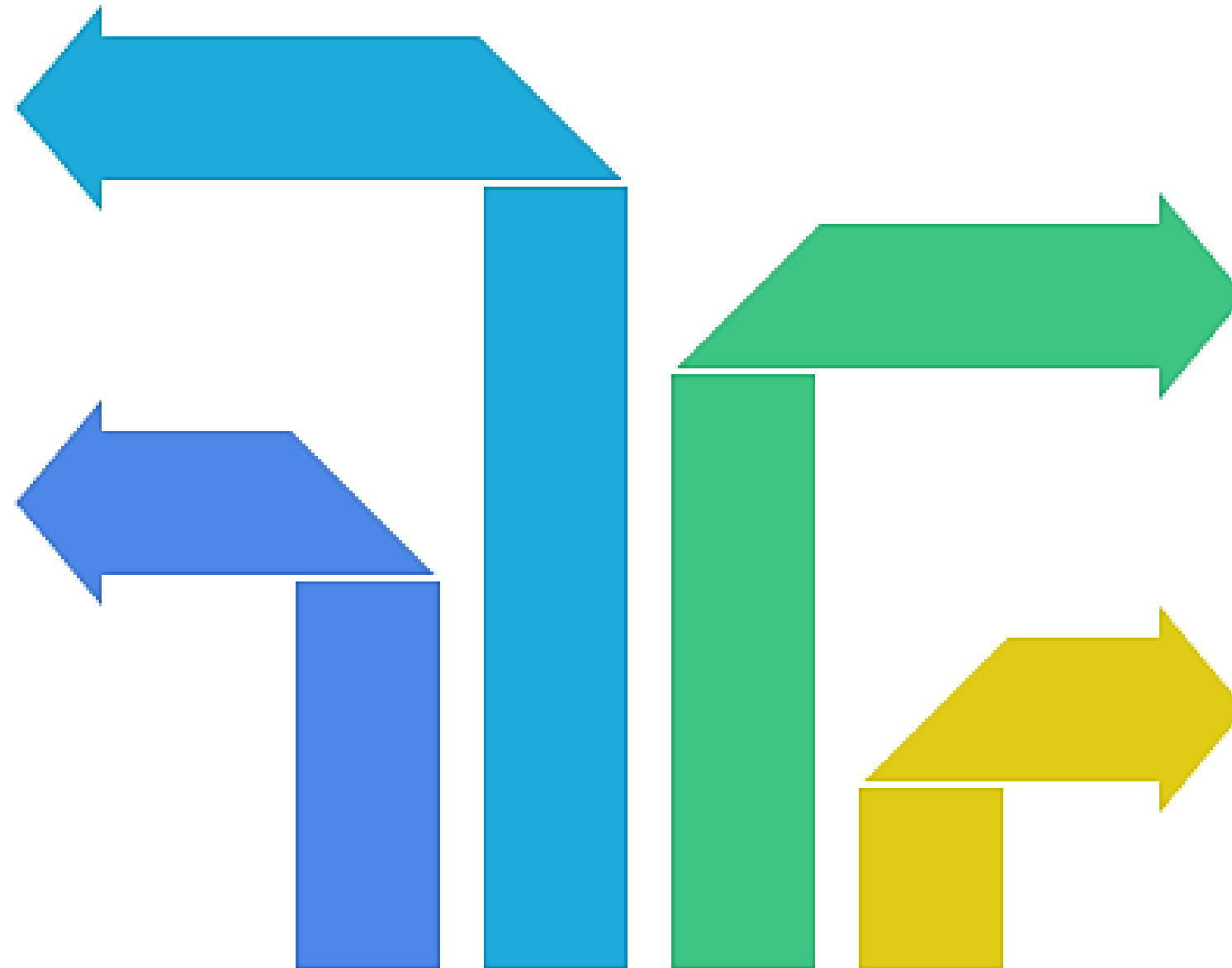
Powerful multi-threaded solution with deep packet inspection, ideal for high-performance environments.

Zeek

Best for threat hunting with behavioral analysis and forensics, requiring technical expertise.

Snort

Simple and widely adopted for small networks, but limited by single-threaded architecture and false positives.



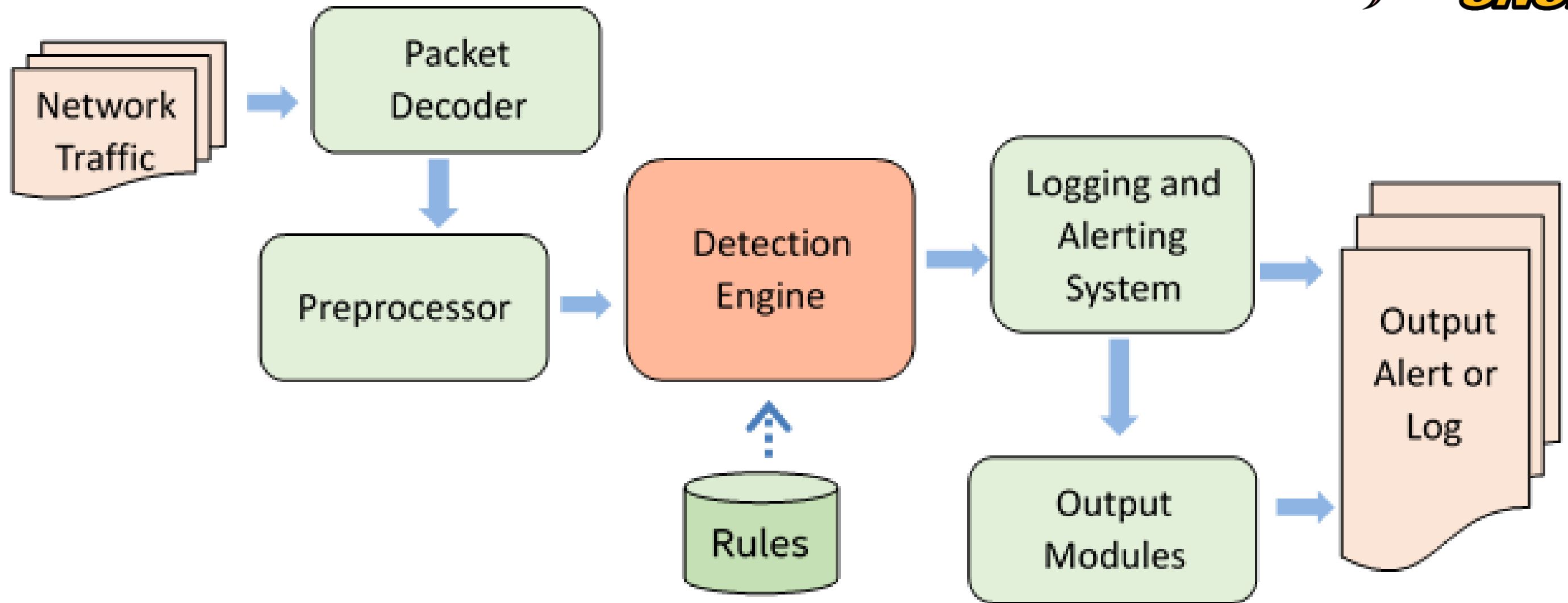
Snort - General Overview



What is Snort?

Snort is an **open-source, rule-based Network Intrusion Detection and Prevention System (NIDS/NIPS)**. It was developed (1999) and still maintained by Martin Roesch, open-source contributors, and the Cisco Talos team.

Snort - Core Components



Snort - Core Components



1. **Packet Decoder:** Decodes network packets based on protocols, preparing them for analysis.

Input (Raw format):

```
00 1B 44 11 3A 4D 00 1B 44 11
3A 4E 08 00 45 00 00 3C 1C 46 40
00 40 06 B1 E6 C0 A8 00 68 C0 A8
00 01
```

Packet Decoder

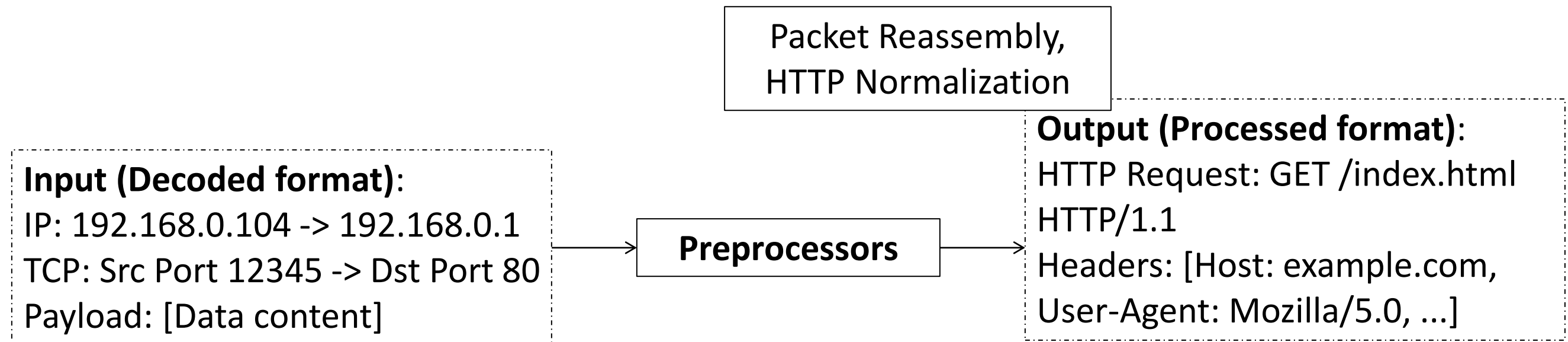
Output (Decoded format):

```
Protocol: TCP
Source IP: 192.168.0.104
Destination IP: 192.168.0.1
Source Port: 12345
Destination Port: 80
Payload: [Data content]
```

Snort - Core Components



2. Preprocessors: Decodes network packets based on protocols, preparing them for analysis.



Snort - Core Components



3. Detection Engine:

Analyzes packets against predefined attack signatures and generates alerts when matches are found.

4. Logging & Alerting System:

Records detected events and generates alerts for administrators to review.

5. Output Plugins:

Sends alerts and logs to external systems like SIEMs or databases for further analysis and action.

Snort - Training Approaches:



Option 1: Manual Installation (Local Setup):

Full control, Realistic setup (installation, configuration, and troubleshooting), ---Requires more time & tech skills.

Option 2: TryHackMe – Snort Room:

Ready-to-use virtual lab environment, Focused on practice rather than setup, Structured learning path, Ideal for quick learning, demos, ---Less exposure to real-world config challenges.

Snort - TryHackMe – Snort Room:



1. How do you access the TryHackMe Snort room?
2. Is there an attacker in the environment?
3. Can we configure Snort?



TryHackMe – Snort Room:

Interactive material and exercise setup

1. Can you extract the IP address of the Snort machine?
2. Can you check which version of Snort is installed?
3. How can you launch an attack on the Snort machine?
4. How can you ensure that the configuration file is valid?
`sudo snort -c /etc/snort/snort.conf -T`

Using Snort:

Sniffer Mode

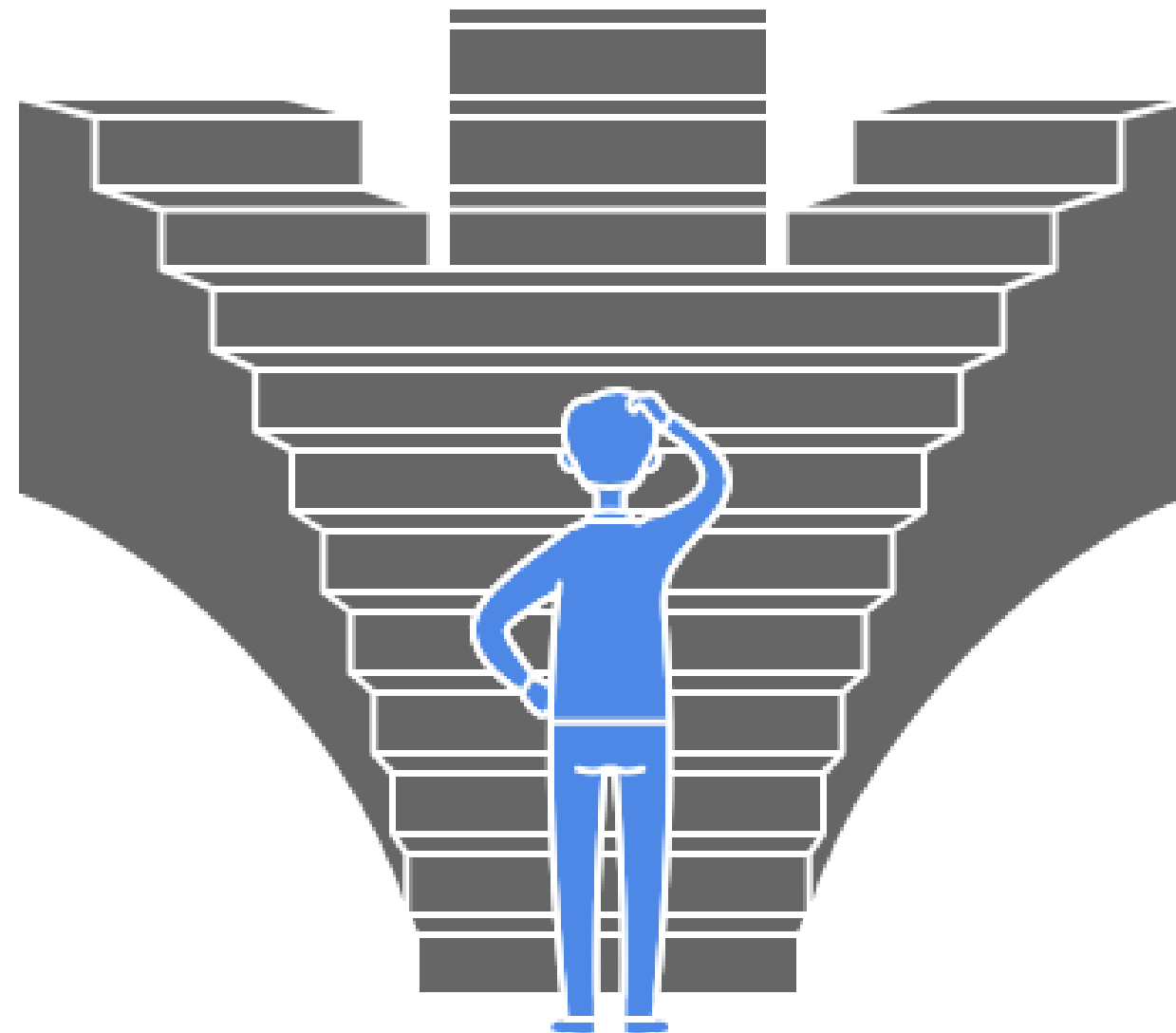
Useful for real-time packet capture and analysis.

Packet Logger Mode

Ideal for offline analysis with logged data.

IDS/IPS Mode

Focuses on intrusion detection with alerts and Reactions



Snort Modes

Snort: Sniffer Mode



The purpose of using Snort's Sniffer Mode is to capture and display raw network traffic in real-time for analysis, without processing or alerting.

But why ?

1. Basic Traffic Inspection (View live Packet Headers)
2. Troubleshooting & Debugging (Verify network communication)
3. Educational Tool



Sniffer Mode : Verbose

```
sudo snort -v -i eth0
```

```
sudo ./traffic-generator.sh
```

WARNING: No preprocessors configured for policy 0.

04/08-17:36:43.590598 10.100.2.28:58426 -> 10.10.21.170:80

TCP TTL:64 TOS:0x0 ID:61963 IpLen:20 DgmLen:68 DF

AP Seq: 0xC5434276 Ack: 0x5730CAA9 Win: 0x101C TcpLen: 32

TCP Options (3) => NOP NOP TS: 778210056 3768344163

Sniffer Mode : dumping packet data mode (-d)



```
sudo snort -d
```

```
sudo ./traffic-generator.sh
```

```
04/08-17:40:26.966395 10.100.2.28:58426 -> 10.10.21.170:80
```

```
TCP TTL:64 TOS:0x0 ID:4988 IpLen:20 DgmLen:68 DF
```

```
***AP*** Seq: 0xC544E52A Ack: 0x5812BC03 Win: 0x101C TcpLen: 32
```

```
TCP Options (3) => NOP NOP TS: 778433434 3768567551
```

```
82 8A D0 12 8E E9 28 12 8E E9 D0 12 8E E8 D1 13 .....(.....
```

Sniffer Mode : link-layer header grabbing (-e)



```
sudo snort -e
```

```
sudo ./traffic-generator.sh
```

```
04/08-17:41:39.412113 02:C8:85:B5:5A:AA -> 02:E9:2C:46:87:AD type:0x800 len:0x42  
10.100.2.28:58426 -> 10.10.21.170:80 TCP TTL:64 TOS:0x0 ID:8309 IpLen:20  
DgmLen:52 DF  
***A*** Seq: 0xC5458B08 Ack: 0x58567B97 Win: 0x101C TcpLen: 32  
TCP Options (3) => NOP NOP TS: 778505880 3768639998
```



Sniffer Mode : full packet dump mode (-X)

```
sudo snort -X
```

```
sudo ./traffic-generator.sh
```

```
04/08-17:44:57.228420 10.100.2.28:58426 -> 10.10.21.170:80
```

```
TCP TTL:64 TOS:0x0 ID:11750 IpLen:20 DgmLen:68 DF
```

```
***AP*** Seq: 0xC54632AC Ack: 0x589DE13F Win: 0x101C TcpLen: 32
```

```
TCP Options (3) => NOP NOP TS: 778703698 3768837811
```

```
0x0000: 02 E9 2C 46 87 AD 02 C8 85 B5 5A AA 08 00 45 00 ..,F.....Z...E.
```

```
0x0010: 00 44 2D E6 40 00 40 06 E0 9A 0A 64 02 1C 0A 0A .D-.@.@....d....
```

```
0x0020: 15 AA E4 3A 00 50 C5 46 32 AC 58 9D E1 3F 80 18 ....P.F2.X..?..
```

```
0x0030: 10 1C CB CB 00 00 01 01 08 0A 2E 6A 13 52 E0 A3 .....j.R..
```

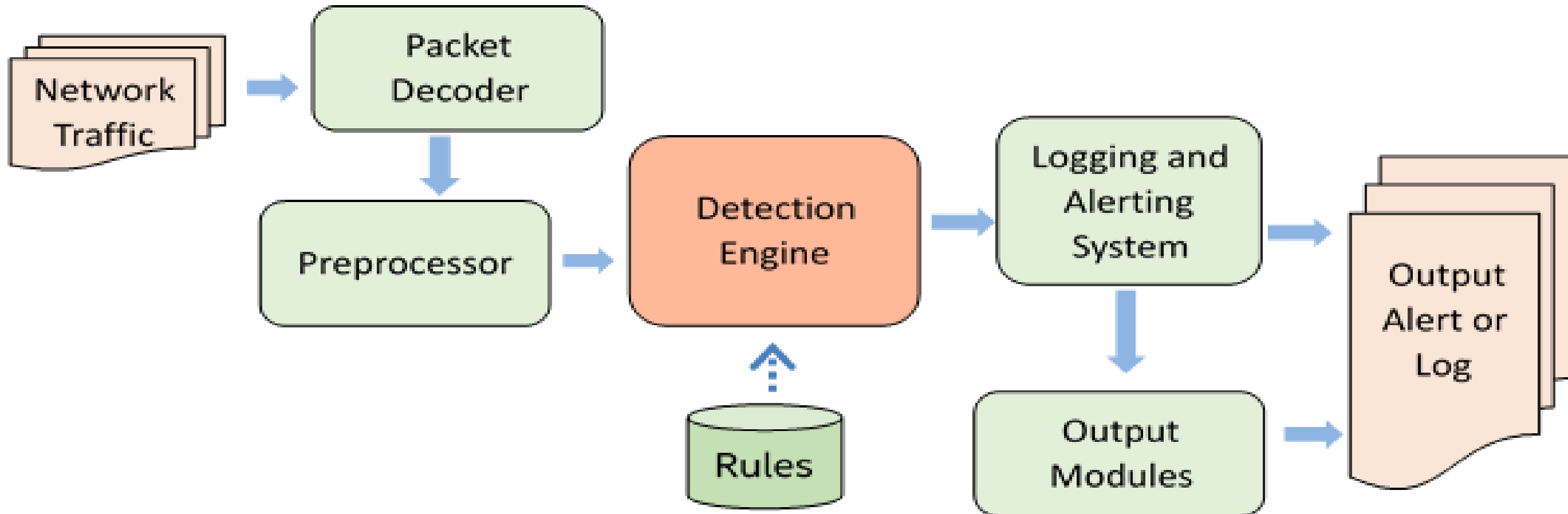
```
0x0040: E6 B3 82 8A 05 34 95 94 FD 34 95 94 05 34 95 95 .....4...4...4..
```

```
0x0050: 04 35
```

Recap



1. NIDS vs NIPS ?
2. Which Tool ?
3. Core components ?



Snort: **Logger Mode**



What it does: Captures packets and saves them to a log file for later analysis.

But why ?

Use case: You want to analyze traffic later, not in real time. Useful for audits or offline forensics.

Snort: Logger Mode



```
sudo snort -dev -l .
```

```
ls
```

```
sudo su
```

```
cat snort....
```

Output: pcap file

Snort: Logger Mode



```
sudo snort -dev -K ASCII -l .
```

```
ls
```

```
cd newfolder
```

```
cat nameOfFile
```

Output: packets in readable text format



Snort: **Logger Mode**

Pourquoi le mode Logger contient deux formats de sortie :
-l et -K ?

1. The binary (pcap) format is ideal for deep analysis with external tools.
2. The ASCII format is better if you just want to read and understand what's going on quickly.



Snort: IDS/IPS Mode

Capabilities of Snort are not limited to sniffing and logging the traffic.

IDS/IPS mode helps you manage the traffic according to user-defined rules.

```
"/etc/snort/rules/local.rules"
```

Snort: IDS/IPS Mode



```
sudo snort -c /etc/snort/snort.conf -A console
```

```
sudo ./traffic-generator.sh
```

1. Run time (seconds)?
2. Snort processed packets ?
3. Pkts/min ?
4. Pkts/sec ?
5. ICMP ?
6. Nbr Alerts ?
7. Nbr Block ?



Snort: IDS/IPS Mode

`sudo snort -c /etc/snort/snort.conf -A fast`

`sudo snort -c /etc/snort/snort.conf -A full`

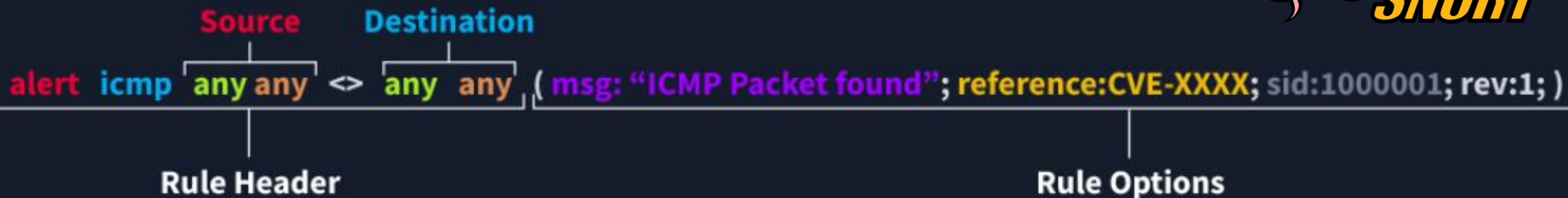
`sudo snort -c /etc/snort/snort.conf -A none`

fast: Fast mode, shows the alert message, timestamp, source and destination ip along with port numbers.

full: Full alert mode, providing all possible information about the alert.

none: Snort will generate a log file.

Rule Structure



Rule Header	alert	Action type
	icmp	Protocol type
	any	Source IP
	any	Source port
	<>	Direction
	any	Destination IP
	any	Destination port

Rule Options	msg	Message
	reference	Reference
	sid	Rule id
	rev	Revision information

Rule Structure: Header



1. Action:

- ❖ Alert
- ❖ Drop
- ❖ Reject

Drop **vs** Reject ?

Drop: Silently drops the packet. No response is sent to the sender.

Reject: Drops the packet and sends an error message to the sender. If it's TCP, sends a TCP RST (reset). If it's UDP/ICMP, sends an ICMP port unreachable.



Rule Structure: Header

2. Protocol: Protocol parameter identifies the type of the protocol that filtered for the rule.

- ❖ Note that Snort2 supports only four protocols filters in the rules (**IP**, **TCP**, **UDP** and **ICMP**).

The fact that Snort only supports 4 protocols in its rules **is not a limitation** of Snort itself, but rather a **logical choice** tied to network architecture and **the layer at which Snort operates**.

The '**protocol**' field in a Snort rule maps to an actual field in the IP header.



Rule Structure: Header

Can **Snort** still **analyze application-layer data** (like **HTTP** payloads, **FTP**, or **SSH**) even though its rules primarily focus on IP, TCP, UDP, and ICMP?

Snort **can detect** the application flows using **port numbers** and **options**. Ex: Snort detects FTP via **TCP port 21**, not direct FTP protocol matching.

What about **HTTP** ? **HTTPS** ?



Rule Structure: Header

Why Snort Only Supports IP, TCP, UDP & ICMP (Despite 150+ Possible Protocols) ?

1. Core Focus on Major Threats – 95%+ of attacks use TCP/UDP (HTTP, SSH, RDP) or ICMP (ping scans).
2. Decoder Limitations – Snort only has built-in decoders for these 4 protocols (required for deep inspection).
3. Performance Priority – Supporting all 150+ protocols would bloat Snort and slow real-time detection.



Rule Structure: Header

3. IP and Port Numbers:

These parameters identify the source and destination IP addresses and associated port numbers filtered for the rule.

```
alert icmp 192.168.1.56 any <> any any (msg: "ICMP Packet From "; sid: 100001; rev:1;)
```

This rule will create an alert for each ICMP packet originating from the 192.168.1.56 IP address. (IP Filtering)



Rule Structure: Header

3. IP and Port Numbers:

```
alert icmp 192.168.1.0/24 any <> any any (msg: "ICMP Packet"; sid: 100001; rev:1;)
```

This rule will create an alert for each ICMP packet originating from the 192.168.1.0/24 subnet. (Filter an IP range)



Rule Structure: Header

3. IP and Port Numbers:

```
alert icmp [192.168.1.0/24, 10.1.1.0/24] any <> any any (msg: "ICMP"; sid: 01; rev:1;)
```

This rule will create an alert for each ICMP packet originating from the 192.168.1.0/24 and 10.1.1.0/24 subnets.



Rule Structure: Header

3. IP and Port Numbers:

```
alert icmp !192.168.1.0/24 any <> any any (msg: "ICMP Packet"; sid: 100001; rev:1;)
```

"negation operator" is used for excluding specific addresses and ports.
Negation operator is indicated with "!"

This rule will create an alert for each ICMP packet not originating from the 192.168.1.0/24 subnet.



Rule Structure: Header

3. IP and Port Numbers:

```
alert tcp any any <> any 21 (msg: "FTP Port 21 Command Activity Detected"; sid:  
100001; rev:1;)
```

This rule will create an alert for each TCP packet sent to port 21.



Rule Structure: Header

3. IP and Port Numbers:

```
alert tcp any any <> any 1:1024 (msg: "TCP 1-1024 System Port Activity"; sid: 100001;  
rev:1;)
```

This rule will create an alert for each TCP packet sent to ports between
1-1024



Rule Structure: Header

3. IP and Port Numbers:

```
alert tcp any any <> any :1024 (msg: "TCP 0-1024 Port Activity"; sid: 100001; rev:1;)
```

This rule will create an alert for each TCP packet sent to ports less than or equal to 1024.

```
alert tcp any any <> any 1025: (msg: "TCP Non-Sys Port Activity"; sid: 100001; rev:1;)
```

This rule will create an alert for each TCP packet sent to source port higher than or equal to 1025.



Rule Structure: Header

3. IP and Port Numbers:

```
alert tcp any any <> any [21,23] (msg: "FTP and Telnet Port 21-23 Activity Detected";  
sid: 100001; rev:1;)
```

This rule will create an alert for each TCP packet sent to port 21 and 23.



Rule Structure: Header

4. Direction:

The direction operator indicates the traffic flow to be filtered by Snort. The left side of the rule shows the source, and the right side shows the destination.

- > Source to destination flow.
- <> Bidirectional flow

Note that **there is no "<-"** operator in Snort.



Rule Structure: Options

There are three main rule options in Snort:

1. General Rule Options
2. Payload Rule Options
3. Non-Payload Detection Rule Options



Rule Structure: Options

1. General Rule Options:

Fundamental rule options for Snort: (msg: " "; sid: ; rev: ;)



Rule Structure: Options

1. General Rule Options:

Msg: Short rule description shown in logs/alerts.

Sid: Unique ID. User rules $\geq 1,000,000$. No overlaps.

Ref: Links to CVEs/external docs for context.

Rev: Tracks rule updates (no backups; manual tracking).

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 1:

“You suspect that HTTP traffic coming from the server 192.168.1.100 is malicious and want Snort to ignore it silently. Write the corresponding Snort rule.”

```
drop tcp 192.168.1.100 any -> any 80 (msg: "Malicious HTTP Traffic  
Blocked"; sid:1000009; rev:1;)
```

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 2:

"You want to generate alerts for all TCP traffic except traffic originating from the internal network 10.0.0.0/24. Write the rule."

```
alert tcp !10.0.0.0/24 any -> any any (msg: "External TCP Traffic  
Detected"; sid:1000010; rev:1;)
```

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 3:

"Write a Snort rule to detect all incoming or outgoing SSH connection attempts (port 22) except those from the admin IP 192.168.1.5."

```
alert tcp !192.168.1.5 any <> any 22 (msg: "Unauthorized SSH Access  
Attempt"; sid:1000011; rev:1;)
```

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 4:

"You want to block and notify about an attacker sending ICMP (ping) requests from IP 203.0.113.42. Write the rule."

```
reject icmp 203.0.113.42 any -> any any (msg: "ICMP Ping Blocked with  
Reject"; sid:1000012; rev:1;)
```

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 5:

"You want to alert on all TCP traffic to system ports (1-1024) except HTTP (80) and HTTPS (443). Write the rule."

```
alert tcp any any -> any [1:79,81:442,444:1024] (msg: "Suspicious System  
Port Activity"; sid:1000015; rev:1;)
```



Rule Structure: Options

There are three main rule options in Snort:

- ~~1. General Rule Options~~
2. Payload Rule Options
3. Non-Payload Detection Rule Options



Rule Structure: Options

2. Payload Rule Options:

Rule options that help to investigate the payload data. These options are helpful to detect specific payload patterns.

- ❖ Content
- ❖ Nocase
- ❖ Fast_pattern



Rule Structure: Options

2. Payload Rule Options:

Content

Purpose : To search for a specific string in the payload (packet content). It is possible to use this option multiple times in a single rule.

Following rules will create an alert for each HTTP packet containing the keyword "GET".

```
alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET";  
sid: 100001; rev:1;)
```

Rule Structure: Options



In the context of malicious communications, such as the **download of a malware**, one possible indicator is an **HTTP request** trying to download a file with the **.exe** extension.

Assuming the attacker uses a GET request, how would you write a Snort rule that detects this behavior by inspecting the HTTP payload?

👉 Hint: You need to look for both "GET" and ".exe" in the payload.

```
alert tcp any any -> any any (msg:"Possible Malware Download";  
content:"GET"; content:".exe"; sid:10000003; rev:1;)
```



Rule Structure: Options

2. Payload Rule Options:

Fast_pattern

Purpose : Prioritise content search to speed up the payload search operation.

```
alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET";  
    fast_pattern; content:".exe"; sid:100001; rev:1;)
```



Rule Structure: Options

2. Payload Rule Options:

Nocase

Purpose : Disabling case sensitivity. Used for enhancing the content searches.

```
alert tcp any any <> any 80 (msg: "GET Request Found"; content:"GET";  
    nocase; sid: 100001; rev:1;)
```

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 6: Detection of C2 (Command & Control) via Suspicious HTTP Requests"

A malware uses HTTP requests to a C2 (Command & Control) server to receive commands. The malicious server uses the URL: <http://evil-c2.com/api/checkin>

Your Tasks:

1. Block all connections to this domain.
2. Log (alert) if an internal client (192.168.1.0/24) attempts to access it.

Hands-On Snort Rule Writing: Real-World Scenarios Challenge



Scenario 6: Detection of C2 (Command & Control) via Suspicious HTTP Requests"

```
drop tcp any any -> any 80 (content:"evil-c2.com"; msg:"C2 Traffic Blocked - evil-c2.com"; sid:1000200; rev:1;)
```

```
alert tcp 192.168.1.0/24 any -> any 80 (content:"evil-c2.com"; msg:"Internal Host Contacting C2 Server"; sid:1000201; rev:1;)
```



Rule Structure: Options

There are three main rule options in Snort:

- ~~1. General Rule Options~~
- ~~2. Payload Rule Options~~
3. Non-Payload Detection Rule Options



Rule Structure: Options

3. Non-Payload Detection Rule Options :

There are rule options that focus on non-payload data. These options will help create specific patterns and identify network issues: **Flags**, **Dsize** (payload size)...

```
alert tcp any any -> any 80 (msg:"SYN scan "; flags:S; sid:1000001;)
```

```
alert ip any any -> any any (msg:"Empty payload "; dsize:0; sid:1000002;)
```

P.S:

To use **Snort with** custom rules containing **actions** like **drop**, **reject**, you need to run it in **inline mode**

```
snort -c /etc/snort/snort.conf -q -Q --daq afpacket -i eth0:eth1 -A console
```

- q** Mode quiet : réduit la quantité de messages affichés (moins verbeux).
- Q** Active le mode inline, ce qui signifie que Snort peut bloquer les paquets en plus de les détecter (fonctionnalité IPS, pas juste IDS).
- daq afpacket** Utilise le DAQ (Data Acquisition) module nommé afpacket, qui permet à Snort de capturer les paquets et de fonctionner en inline de façon performante sur Linux.